

Практическое занятие 19. Настройка маршрутов (URLs) и обработка запросов

Цель занятия:

Научиться настраивать маршруты URL для различных страниц приложения `events`, создавать простые представления (views) и шаблоны для отображения контента. Дополнительно выполнить задания по созданию приложений `about` и `contact` с базовыми страницами.

Задачи занятия

1. Введение в маршрутизацию Django
 2. Добавление основных маршрутов в `events/urls.py`
 3. Создание простых представлений в `events/views.py`
 4. Создание базовых HTML-шаблонов
 5. Тестирование настроенных маршрутов и представлений
 6. Выполнение практического упражнения. Добавление нового маршрута и представления
 7. Выполнение дополнительных заданий. Создание приложений `about` и `contact`
-

Задача 1: Введение в маршрутизацию Django

1.1. Что такое маршрутизация в Django?

- **Маршрутизация (URL routing)** — это механизм сопоставления URL-адресов с соответствующими представлениями (views) в Django.
- Позволяет определить, какая часть кода будет обрабатывать тот или иной запрос от пользователя.

1.2. Основные компоненты маршрутизации:

- **URLConf (URL Configuration):** Файлы `urls.py`, где определяются маршруты.
 - **Представления (Views):** Функции или классы, которые обрабатывают запросы и возвращают ответы.
 - **Маршруты (Paths):** Правила сопоставления URL с представлениями.
-

Задача 2: Добавление основных маршрутов в `events/urls.py`

2.1. Создание файла `urls.py` в приложении `events` (если он еще не создан):

1. В VSCode перейдите в папку `events/`.
2. Если файла `urls.py` нет, создайте его: Правый клик по папке `events` > **New File** > назвать `urls.py`.

2.2. Добавление маршрутов:

Вставьте следующий код в `events/urls.py`:

```
from django.urls import path
from . import views

app_name = 'events'

urlpatterns = [
    path('', views.home, name='home'),
    path('events/', views.event_list, name='event_list'),
    path('events/<int:event_id>/', views.event_detail, name='event_detail'),
]
```

Пояснение маршрутов:

- `''` : Главная страница приложения `events`.
- `'events/'` : Страница со списком мероприятий.
- `'events/<int:event_id>/'` : Страница с деталями конкретного мероприятия по его ID.

Задача 3: Создание простых представлений в `events/views.py`

3.1. Открытие файла `events/views.py`:

- В папке `events/` откройте файл `views.py`.

3.2. Добавление функций-представлений:

Добавьте следующие функции в `views.py`:

```
from django.shortcuts import render
```

```
def home(request):
    return render(request, 'events/home.html')

def event_list(request):
    # В будущем здесь будет вывод списка мероприятий
    return render(request, 'events/event_list.html')

def event_detail(request, event_id):
    # В будущем здесь будет вывод деталей мероприятия
    return render(request, 'events/event_detail.html', {'event_id':
event_id})
```

Пояснение:

- **home:** Отображает главную страницу приложения `events`.
- **event_list:** Отображает страницу со списком мероприятий.
- **event_detail:** Отображает страницу с деталями конкретного мероприятия по его ID.

Задача 4: Создание базовых HTML-шаблонов

4.1. Создание директории для шаблонов (если она еще не создана):

1. В корневой директории проекта (на одном уровне с `manage.py`) создайте папку `templates`, если ее нет.
2. Внутри `templates/` создайте папку `events`.

4.2. Создание шаблонов:

1. Шаблон `home.html`:

Измените файл `home.html` в `templates/events/` и добавьте следующий код:

```
<!DOCTYPE html>
<html>
<head>
    <title>Главная страница</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Добро пожаловать на сайт мероприятий!</h1>
    <nav>
        <a href="{% url 'events:event_list' %}">Список мероприятий</a>
```

```
|  
    </nav>  
</body>  
</html>
```

Если вы выполнили дополнительные задания, добавьте

```
<a href="{% url 'about:about' %}">О нас</a> |  
<a href="{% url 'contact:contact' %}">Контакты</a>
```

2. Шаблон event_list.html :

Создайте файл event_list.html в templates/events/ и добавьте следующий код:

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Список мероприятий</title>  
    {% load static %}  
    <link rel="stylesheet" type="text/css" href="{% static  
'events/css/styles.css' %}">  
</head>  
<body>  
    <h1>Список мероприятий</h1>  
    <p>Здесь будет отображаться список всех мероприятий.</p>  
    <a href="{% url 'events:home' %}">Вернуться на главную</a>  
</body>  
</html>
```

3. Шаблон event_detail.html :

Создайте файл event_detail.html в templates/events/ и добавьте следующий код:

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Детали мероприятия</title>  
    {% load static %}  
    <link rel="stylesheet" type="text/css" href="{% static  
'events/css/styles.css' %}">  
</head>  
<body>  
    <h1>Детали мероприятия</h1>  
    <p>Информация о мероприятии с ID: {{ event_id }}</p>  
    <a href="{% url 'events:event_list' %}">Вернуться к списку
```

```
мероприятий</a>
</body>
</html>
```

Пояснение:

- **home.html:** Главная страница с навигацией по сайту.
- **event_list.html:** Страница для отображения списка мероприятий (пока статическая).
- **event_detail.html:** Страница для отображения деталей конкретного мероприятия по его ID (пока статическая).

Задача 5: Тестирование настроенных маршрутов и представлений

5.1. Настройка файла `event_manager/urls.py`:

Убедитесь, что в `event_manager/urls.py` подключены маршруты приложения `events`, а также маршруты для приложений `about` и `contact` (если они вами создавались в ходе выполнения дополнительных заданий):

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('events.urls', namespace='events')),
    # about только в случае выполнения дополнительных заданий
    path('about/', include('about.urls', namespace='about')),
    path('contact/', include('contact.urls',
        namespace='contact')),
    # contact только в случае выполнения дополнительных заданий
]
```

5.2. Запуск сервера разработки:

1. Активируйте виртуальное окружение (если не активировано):

- На Windows:

```
venv\Scripts\activate
```

- На macOS/Linux:

```
source venv/bin/activate
```

2. Запустите сервер:

```
python manage.py runserver
```

- **Примечание:** Если команда `python` не работает на macOS/Linux, используйте `python3`.

5.3. Проверка маршрутов в браузере:

1. Главная страница (`home`):

Перейдите по адресу <http://127.0.0.1:8000/>

- **Ожидаемый результат:** Страница с заголовком "Добро пожаловать на сайт мероприятий!" и навигацией.

2. Список мероприятий (`event_list`):

Перейдите по адресу <http://127.0.0.1:8000/events/>

- **Ожидаемый результат:** Страница с заголовком "Список мероприятий" и сообщением о будущей динамике.

3. Детали мероприятия (`event_detail`):

Перейдите по адресу <http://127.0.0.1:8000/events/1/>

- **Ожидаемый результат:** Страница с заголовком "Детали мероприятия" и информацией о мероприятии с ID 1.

5.4. Устранение возможных ошибок:

- **404 Ошибка:** Если страница не найдена, убедитесь, что маршруты и представления настроены правильно.
- **Ошибки в шаблонах:** Проверьте синтаксис HTML и правильность путей к статическим файлам.

Задача 6: Практическое упражнение: Добавление нового маршрута и представления

6.1. Добавление нового маршрута:

1. Откройте файл `events/urls.py`.
2. Добавьте новый маршрут для страницы "Контакты":

```
path('events/<int:event_id>/delete/', views.delete_event,
name='delete_event'),
```

Полный список маршрутов:

```
from django.urls import path
from . import views

app_name = 'events'

urlpatterns = [
    path('', views.home, name='home'),
    path('events/', views.event_list, name='event_list'),
    path('events/<int:event_id>/', views.event_detail,
name='event_detail'),
    path('events/<int:event_id>/delete/', views.delete_event,
name='delete_event'),
]
```

6.2. Создание представления `delete_event`:

1. Откройте файл `events/views.py`.
2. Добавьте функцию-представление:

```
from django.shortcuts import render, get_object_or_404, redirect

def delete_event(request, event_id):
    if request.method == 'POST':
        # Пока не реализовано удаление, просто перенаправляем на список
        return redirect('events:event_list')
    return render(request, 'events/delete_event.html', {'event_id':
event_id})
```

6.3. Создание шаблона `delete_event.html`:

1. Создайте файл `delete_event.html` в `templates/events/`.
2. Добавьте следующий код:

```
<!DOCTYPE html>
<html>
<head>
    <title>Удалить мероприятие</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
```

```
'events/css/styles.css' %}">
</head>
<body>
  <h1>Удалить мероприятие</h1>
  <p>Вы уверены, что хотите удалить мероприятие с ID: {{ event_id }}?
</p>
  <form method="post">
    {% csrf_token %}
    <button type="submit">Да, удалить</button>
    <a href="{% url 'events:event_detail' event_id=event_id
%}">Отмена</a>
  </form>
</body>
</html>
```

6.4. Тестирование:

1. Перейдите по адресу <http://127.0.0.1:8000/events/1/delete/> (замените 1 на существующий event_id).
2. **Ожидаемый результат:** Страница подтверждения удаления мероприятия.
3. **Проверка:** Нажмите кнопку "Да, удалить" — должно произойти перенаправление на список мероприятий.

6.5. Обсуждение:

- Пока функция `delete_event` не реализует фактическое удаление, это подготовительный шаг для будущих занятий.
- Познакомились с обработкой POST-запросов и использованием CSRF-токенов в формах.

Задача 7: Дополнительные задания: Создание приложений `about` и `contact`

Цель:

Расширить функциональность проекта, добавив приложения `about` и `contact`, которые будут содержать страницы "О нас" и "Контакты" соответственно.

Задание 7.1: Создание приложения `about`

1. Создание приложения `about` :

В терминале, находясь в корневой директории проекта, выполните команду:

```
python manage.py startapp about
```

2. Регистрация приложения `about` в проекте:

1. Откройте файл `event_manager/settings.py`.
2. Добавьте `'about'` в список `INSTALLED_APPS`:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'events',  
    'about', # Добавлено приложение about  
]
```

3. Сохраните изменения (`Ctrl+S` или `Cmd+S`).

3. Настройка маршрутов для приложения `about`:

1. Создайте файл `about/urls.py` и добавьте следующий код:

```
from django.urls import path  
from . import views  
  
app_name = 'about'  
  
urlpatterns = [  
    path('', views.about, name='about'),  
]
```

2. Подключите маршруты приложения `about` в `event_manager/urls.py`:

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('events.urls', namespace='events')),  
    path('about/', include('about.urls', namespace='about')),  
    path('contact/', include('contact.urls', namespace='contact')),  
]
```

3. ****Сохраните изменения** (Ctrl+S или Cmd+S).

4. Создание представления `about` :

Откройте файл `about/views.py` и добавьте функцию:

```
from django.shortcuts import render

def about(request):
    return render(request, 'about/about.html')
```

5. Создание шаблона `about.html` :

1. **Создайте директорию** `templates/about/` **внутри проекта.**
2. **В** `templates/about/` **создайте файл** `about.html` **со следующим содержимым:**

```
<!DOCTYPE html>
<html>
<head>
    <title>О нас</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>О нашей компании</h1>
    <p>Здесь вы можете разместить информацию о вашей компании, миссии,
команде и т.д.</p>
    <a href="{% url 'events:home' %}">Вернуться на главную страницу</a>
</body>
</html>
```

6. Тестирование:

- Запустите сервер (если он не запущен):

```
python manage.py runserver
```

- Перейдите по адресу <http://127.0.0.1:8000/about/>.
- Убедитесь, что страница "О нас" отображается корректно.

Задание 7.2: Создание приложения `contact`

1. Создание приложения `contact` :

В терминале, находясь в корневой директории проекта, выполните команду:

```
python manage.py startapp contact
```

2. Регистрация приложения `contact` в проекте:

1. Откройте файл `event_manager/settings.py` .
2. Добавьте `'contact'` в список `INSTALLED_APPS` :

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'events',  
    'about',  
    'contact', # Добавлено приложение contact  
]
```

3. Сохраните изменения (`Ctrl+S` или `Cmd+S`).

3. Настройка маршрутов для приложения `contact` :

1. Создайте файл `contact/urls.py` и добавьте следующий код:

```
from django.urls import path  
from . import views  
  
app_name = 'contact'  
  
urlpatterns = [  
    path('', views.contact, name='contact'),  
]
```

2. Подключите маршруты приложения `contact` в `event_manager/urls.py` :

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('events.urls', namespace='events')),  
]
```

```
path('about/', include('about.urls', namespace='about')),
path('contact/', include('contact.urls', namespace='contact')),
]
```

3. ****Сохраните изменения (Ctrl+S или Cmd+S).**

4. Создание представления `contact`:

Откройте файл `contact/views.py` и добавьте функцию:

```
from django.shortcuts import render

def contact(request):
    return render(request, 'contact/contact.html')
```

5. Создание шаблона `contact.html`:

1. **Создайте директорию `templates/contact/` внутри проекта.**
2. **В `templates/contact/` создайте файл `contact.html` со следующим содержимым:**

```
<!DOCTYPE html>
<html>
<head>
    <title>Контакты</title>
    {% load static %}
    <link rel="stylesheet" type="text/css" href="{% static
'events/css/styles.css' %}">
</head>
<body>
    <h1>Свяжитесь с нами</h1>
    <p>Здесь вы можете разместить контактную информацию: адрес,
телефон, email и т.д.</p>
    <a href="{% url 'events:home' %}">Вернуться на главную страницу</a>
</body>
</html>
```

6. Тестирование:

- Запустите сервер (если он не запущен):

```
python manage.py runserver
```

- Перейдите по адресу <http://127.0.0.1:8000/contact/>.
- Убедитесь, что страница "Контакты" отображается корректно.

Дополнительные подсказки по меню и терминалам

Терминал в Visual Studio Code (VSCode)

Русский:

- **Открытие терминала:**
 - Перейдите в меню **Вид > Терминал** или используйте сочетание клавиш `Ctrl+``.
- **Завершение работы терминала:**
 - Нажмите иконку **Trash** (мусорное ведро) в верхней части терминала или перейдите в меню **Терминал > Завершить терминал**.

English:

- **Opening the Terminal:**
 - Go to **View > Terminal** or use the shortcut `Ctrl+``.
- **Killing the Terminal:**
 - Click the **Trash** icon at the top of the terminal or go to **Terminal > Kill Terminal**.

Интерфейс VSCode

Русский:

- **Панель расширений:** Слева нажмите на иконку **Расширения** или используйте `Ctrl+Shift+X`.
- **Панель поиска файлов:** Используйте сочетание клавиш `Ctrl+P` для быстрого поиска файлов.
- **Панель управления Git:** Нажмите на иконку **Source Control** (иконка с веткой) или используйте `Ctrl+Shift+G`.

English:

- **Extensions Panel:** Click on the **Extensions** icon on the left or use `Ctrl+Shift+X`.
- **File Search Panel:** Use the shortcut `Ctrl+P` to quickly search for files.
- **Git Control Panel:** Click on the **Source Control** icon (branch icon) on the left or use `Ctrl+Shift+G`.

Общие советы по работе с терминалом в VSCode

- **Навигация по терминалу:**
 - Используйте стрелки вверх и вниз для просмотра предыдущих команд.
- **Множественные терминалы:**

- Вы можете открыть несколько терминалов одновременно, нажав на кнопку **+** в верхней части панели терминала.
 - **Переключение между терминалами:**
 - Используйте выпадающее меню рядом с кнопкой **+** для переключения между открытыми терминалами.
-

Заключение

Поздравляем! Вы успешно:

- Настроили маршруты URL для приложения `events`.
 - Создали простые представления и соответствующие HTML-шаблоны.
 - Добавили и настроили дополнительные приложения `about` и `contact` с базовыми страницами.
 - Поняли основы маршрутизации и обработки запросов в Django.
-

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться к преподавателю или обсуждать с одногруппниками.

Успехов в разработке!

Вопросы для обсуждения

- **Что делать, если маршрут не работает?**
 - Проверьте синтаксис в `urls.py`.
 - Убедитесь, что соответствующее представление (`view`) существует и правильно подключено.
 - Проверьте, активировано ли виртуальное окружение и запущен ли сервер.
 - **Как добавить новые маршруты и представления?**
 - Следуйте аналогичной структуре: добавьте маршрут в `urls.py`, создайте соответствующее представление в `views.py`, и создайте шаблон для отображения.
 - **Как организовать шаблоны для разных приложений?**
 - Используйте директории внутри `templates/` для каждого приложения, например, `templates/events/`, `templates/about/`, `templates/contact/`.
-

Полезные ресурсы

- [Документация Django](#)
- [Документация Django REST Framework](#)
- [Документация Git](#)
- [Документация GitHub](#)
- [Django Templates](#)
- [Django Static Files](#)
- [Документация VSCode](#)
- [Simple JWT \(JSON Web Token\) Documentation](#)